LABORATÓRIO 01

Características de repositórios populares

João Pedro Campos

Lívia Bontempo Leite

Pontifícia Universidade Católica de Minas Gerais
Engenharia de Software

Laboratório de Experimentação de Software

Belo Horizonte

2026

Estrutura de Documento de Experimento

1. Introdução

1.1 Contextualização

O ecossistema de software de código aberto tornou-se um pilar fundamental da tecnologia moderna e é usada como base para inúmeras aplicações comerciais e projetos de infraestrutura digital. Plataformas como o GitHub centralizam a colaboração de milhões de desenvolvedores, e a popularidade de um repositório, frequentemente mensurada pelo número de "estrelas", funciona como um indicador da sua relevância pela comunidade.

No entanto, apenas a popularidade por si só é uma métrica ineficiente. Com isso, para compreender os fatores que sustentam o sucesso e longevidade de um projeto open-source, é essencial analisar suas características intrínsecas de desenvolvimento e manutenção. Neste prisma, o estudo propõe uma investigação quantitativa sobre os 1.000 repositórios com maior número de estrelas no GitHub, com o objetivo de identificar padrões relacionados à maturidade do projeto, ao volume de colaboração externa, à frequência de atualizações e à governança do ciclo de vida do software.

Essa análise busca validar empiricamente se a percepção comum sobre projetos maduros, ativamente mantidos e abertos à comunidade, se sustentam em dados concretos, fornecendo insights valiosos para desenvolvedores, gestores de comunidades e pesquisadores da área de engenharia de software.

1.2 Problema Foco do Experimento

O problema específico que este experimento busca investigar é: repositórios populares do GitHub, definidos como os 1.000 com maior número de estrelas, compartilham padrões mensuráveis de maturidade, colaboração externa, frequência de lançamentos, atividade de manutenção, escolha de linguagem de programação e resolução de issues? Em outras palavras, busca-se determinar se a popularidade de um projeto de código aberto está sistematicamente associada a um conjunto de características de processo e produto que possam ser objetivamente quantificadas a partir de dados extraídos da API do GitHub.

1.3 Questões-problema ou Questões-Pesquisa

As principais questões que guiam este estudo são:

- RQ01. Sistemas populares são maduros/antigos?
- RQ02. Sistemas populares recebem muita contribuição externa?
- RQ03. Sistemas populares lançam releases com frequência?
- RQ04. Sistemas populares são atualizados com frequência?
- RQ05. Sistemas populares são escritos nas linguagens de programação mais populares?
- RQ06. Sistemas populares possuem um alto percentual de issues fechadas?

1.4 Hipótese(s)

Com base no problema e nas questões de pesquisa, a(s) hipótese(s) formulada(s) para este experimento é(são):

Hipótese Principal: Repositórios populares do GitHub apresentam, de forma consistente, um conjunto de características positivas de manutenção e colaboração: são projetos mais antigos e maduros, possuem alto volume de contribuições externas via pull requests aceitos, realizam lançamentos de versões com maior frequência, são atualizados recentemente, utilizam linguagens de programação amplamente difundidas no mercado e apresentam alta taxa de resolução de issues.

Hipóteses Secundárias:

- **H1:** Repositórios populares tendem a ser projetos de longa data, pois confiança e a estabilidade necessárias para atrair uma grande base de usuários são construídas ao longo de anos. Projetos novos, normalmente, passam por um período de maturação antes de alcançarem uma popularidade massiva. Com isso, espera-se encontrar uma mediana de idade significativamente superior a um ciclo de desenvolvimento curto.
- **H2:** Sistemas populares recebem muita contribuição externa. Acredita-se que um número elevado de pull requests aceitos é um forte indicador de uma comunidade ativa e saudável. A capacidade de atrair e integrar colaborações da comunidade é um fator determinante para a escalabilidade e sustentabilidade do projeto, o que atrai talentos que irão acelerar a inovação e

a correção de falhas. Portanto, espera-se uma mediana elevada no número de pull requests aceitos.

- **H3:** Projetos populares tendem a formalizar entregas através de releases, que marcam novas versões com funcionalidades e correções, o que sinaliza governança e profissionalismo, pois fornece aos usuários pontos de referência estáveis e um registro claro das evoluções. Espera-se um número significativo de releases, indicando um ciclo de desenvolvimento ativo.
- **H4:** Sistemas populares são atualizados com frequência. A manutenção contínua é crucial para a relevância e segurança de um projeto. Para manter isso, esses projetos precisam responder rapidamente a vulnerabilidades e se adaptar a novas versões de dependências. A expectativa é que a mediana de tempo desde a última atualização seja muito baixa (poucos dias), refletindo atividade constante.
- **H5:** Sistemas populares são escritos nas linguagens mais populares. A popularidade de um projeto pode ser influenciada pela popularidade da sua linguagem principal, pois isso afeta a quantidade de desenvolvedores capazes de usá-lo e contribuir. Linguagens como Python, JavaScript/TypeScript e Go são esperadas no topo do ranking devido aos seus vastos ecossistemas, aplicabilidade em domínios de alta demanda (IA, desenvolvimento web, cloud) e grande base de desenvolvedores, o que facilita a adoção e a contribuição.
- **H6:** Sistemas populares possuem um alto percentual de issues fechadas. Uma alta taxa de issues fechadas sugere uma boa capacidade de resposta da equipe de manutenção para resolver bugs, responder dúvidas e gerenciar o projeto. Uma gestão eficaz de issues (que incluem bugs, sugestões de melhoria e dúvidas) é crucial para a percepção de um projeto bem cuidado e responsivo. Além disso, um acúmulo de issues abertas pode sinalizar problemas de manutenção. Portanto, esperamos encontrar uma mediana da razão de issues fechadas superior a 80%.

1.5 Objetivo (principal e específicos)

Objetivo Principal:

O propósito central desse experimento é caracterizar quantitativamente os 1.000 repositórios mais populares do GitHub, por meio da extração e análise de métricas relacionadas à maturidade, colaboração, manutenção e tecnologia adotada, com o intuito de identificar padrões comuns que possam explicar ou acompanhar a popularidade de projetos de software de código aberto.

Objetivos Específicos:

1. Coletar automaticamente, via API GraphQL do GitHub, os dados estruturais dos 1.000 repositórios com maior número de estrelas, abrangendo campos como data de criação, data de atualização, linguagem primária, pull requests aceitos, total de releases, stargazers e contagem de issues abertas e fechadas.

2. Analisar estatisticamente os dados por meio de medidas de tendência central (especificamente a mediana, por ser robusta a outliers) para sumarizar o comportamento de cada métrica no conjunto amostral.

3. Visualizar os resultados por meio de gráficos e tabelas gerados automaticamente, possibilitando a interpretação visual dos padrões identificados e a comunicação clara das respostas às questões de pesquisa.

4. Responder empiricamente cada uma das seis questões de pesquisa, validando ou refutando as hipóteses formuladas com base nos dados reais extraídos da plataforma

2. Metodologia

2.1 Passo a passo do experimento

O experimento foi conduzido seguindo a seguinte sequência de ações:

Preparação:
- Definição do objeto de estudo: repositórios públicos do GitHub ordenados por popularidade (estrelas).
- Configuração do ambiente em Python com bibliotecas requests, python-dotenv e matplotlib.
- Geração e configuração de token de acesso da API do GitHub em arquivo .env.
- Definição de parâmetros de coleta no projeto: endpoint GraphQL, total alvo de 1.000 repositórios e paginação em lotes de 10 itens.

Execução:
- Coleta automatizada via API GraphQL do GitHub com a consulta stars:>0 sort:stars-desc.
- Navegação paginada usando endCursor/hasNextPage até atingir o total planejado.
- Extração dos atributos necessários às RQs (createdAt, updatedAt, pullRequests MERGED, releases, issues, closedIssues, primaryLanguage, stargazerCount).
- Transformação dos dados para métricas derivadas: idade em dias, dias desde última atualização e razão de issues fechadas.
- Persistência dos resultados em arquivo CSV para análise posterior.

Monitoramento:
- Acompanhamento contínuo por logs de progresso no terminal (quantidade coletada por página).
- Tratamento de falhas de rede e API com retentativas controladas (até 5 tentativas) e espera incremental.
- Validação de consistência mínima: campos ausentes tratados de forma segura e divisão por zero evitada no cálculo da razão de issues.

Finalização:
- Consolidação das estatísticas descritivas por mediana para as RQs.
- Geração de visualizações e relatório em HTML com gráficos e tabelas.
- Exportação de resultados estruturados em JSON para rastreabilidade e reuso.

2.2 Decisões

As principais decisões metodológicas tomadas incluem:

- **Amostragem:** Amostra não probabilística intencional composta pelos 1.000 repositórios mais populares do GitHub, usando estrelas como critério objetivo de seleção.

- **Controle de Variáveis:** Fonte única de dados, mesma consulta, mesmos campos e mesma lógica de extração para todos os repositórios, mesmo horizonte temporal de cálculo (uso de um instante de coleta para derivar idade e última atualização), mesma métrica de tendência central (mediana).

- **Duração:** Duração variável conforme latência da API, limites de requisição e retentativas.

2.3 Materiais utilizados

Os materiais e recursos empregados no experimento estão listados abaixo:

- Linguagem e ambiente: Python 3.x em ambiente virtual.
- Bibliotecas: requests (requisições HTTP), python-dotenv (configuração segura do token), matplotlib (visualizações).
- Infraestrutura de dados: GitHub GraphQL API, token de acesso pessoal, arquivos de saída CSV, JSON e HTML.

2.4 Métodos utilizados

Para a execução e análise, foram utilizados os seguintes métodos:

- **Método de Coleta de Dados:** Medição automatizada por mineração de repositórios de software (MSR), com consulta programática à API GraphQL do GitHub e coleta paginada.

- **Método de Análise Estatística:** Estatística descritiva com foco em mediana para variáveis numéricas (idade, PRs aceitos, releases, tempo de atualização e razão de issues fechadas).

2.5 Métricas e suas Unidades

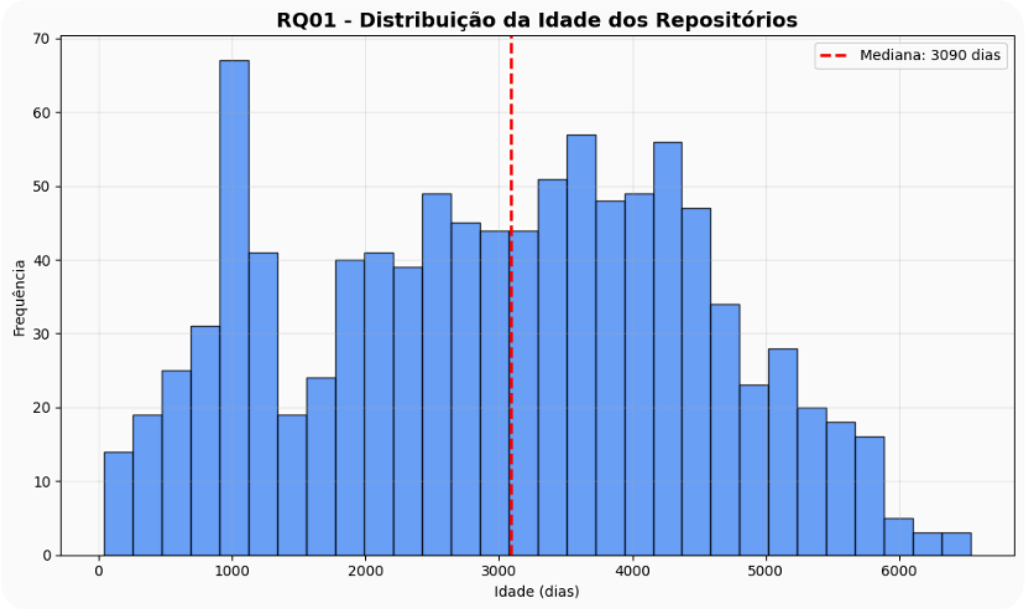
As métricas utilizadas para medir o sucesso ou o resultado do experimento, juntamente com suas unidades de medida, são:

Métrica	Unidade	Descrição
Idade do repositório	Dias	Tempo entre a data de criação do repositório e a data da coleta dos dados. Utilizada para avaliar a maturidade do projeto.
Pull Requests aceitos	Quantidade	Número total de pull requests que foram aceitos (merged) no repositório, indicando o nível de contribuição da comunidade.

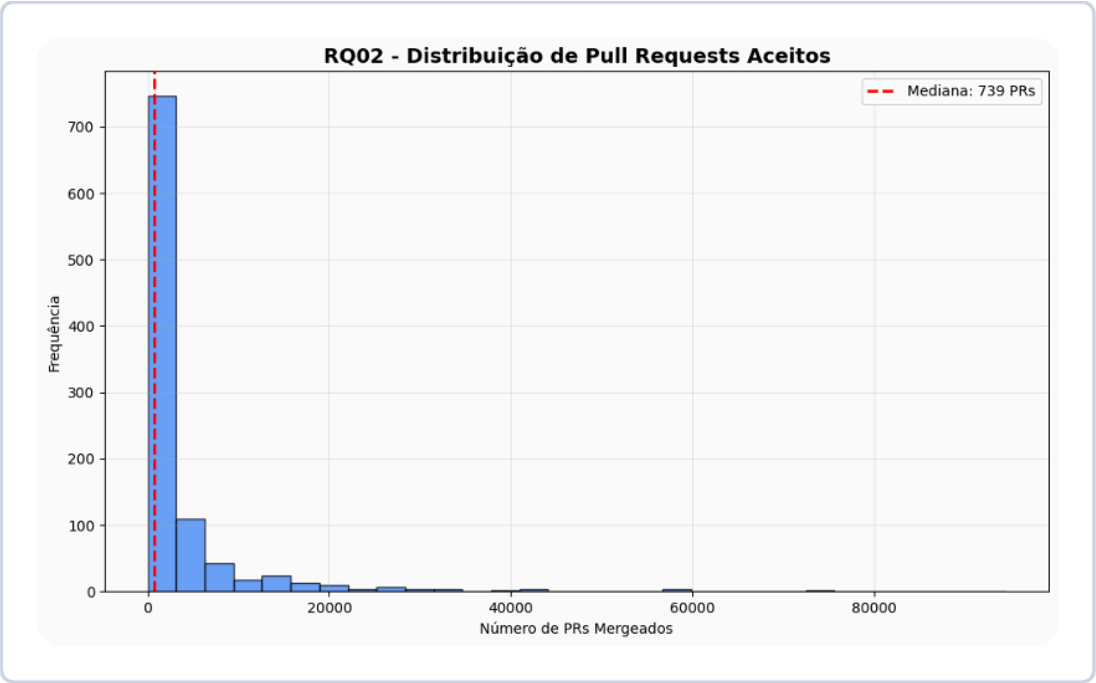
Métrica	Unidade	Descrição
Número de Releases	Quantidade	Total de versões lançadas oficialmente no repositório, indicando frequência de entrega e organização do ciclo de desenvolvimento.
Dias desde a última atualização	Dias	Tempo entre a última atividade de commit e a data da coleta. Utilizada para avaliar a frequência de atualização/manutenção.
Linguagem de Programação Primária	Percentual	Linguagem com maior percentual de código no repositório. Utilizada para avaliar a popularidade tecnológica.
Razão de Issues Fechadas	Percentual	Proporção entre o número de issues fechadas e o número total de issues (abertas + fechadas), para avaliar a capacidade de resposta e gestão de issues.

3. Visualização dos Resultados

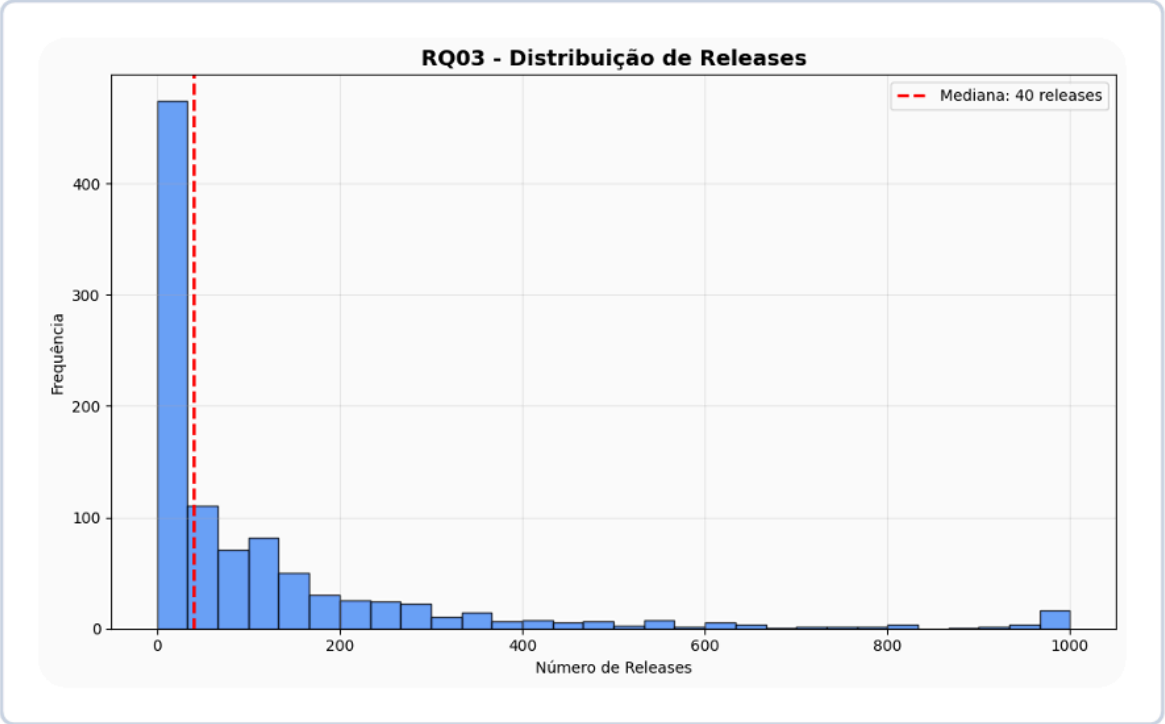
3.1 Visualização tabular, lista ou gráfica do que foi colhido no experimento



Descrição do gráfico:O histograma da idade dos repositórios apresenta a distribuição da quantidade de dias desde a criação de cada projeto até o momento da coleta dos dados. Observa-se uma distribuição relativamente ampla, com projetos variando desde alguns meses até mais de 6000 dias de existência. A linha tracejada vermelha representa a mediana da distribuição, posicionada em aproximadamente 3090 dias (cerca de 8,5 anos). Esse resultado sugere que os repositórios mais populares do GitHub tendem a ser projetos relativamente antigos, indicando que a popularidade geralmente é construída ao longo do tempo, à medida que o projeto amadurece e consolida sua base de usuários e colaboradores.

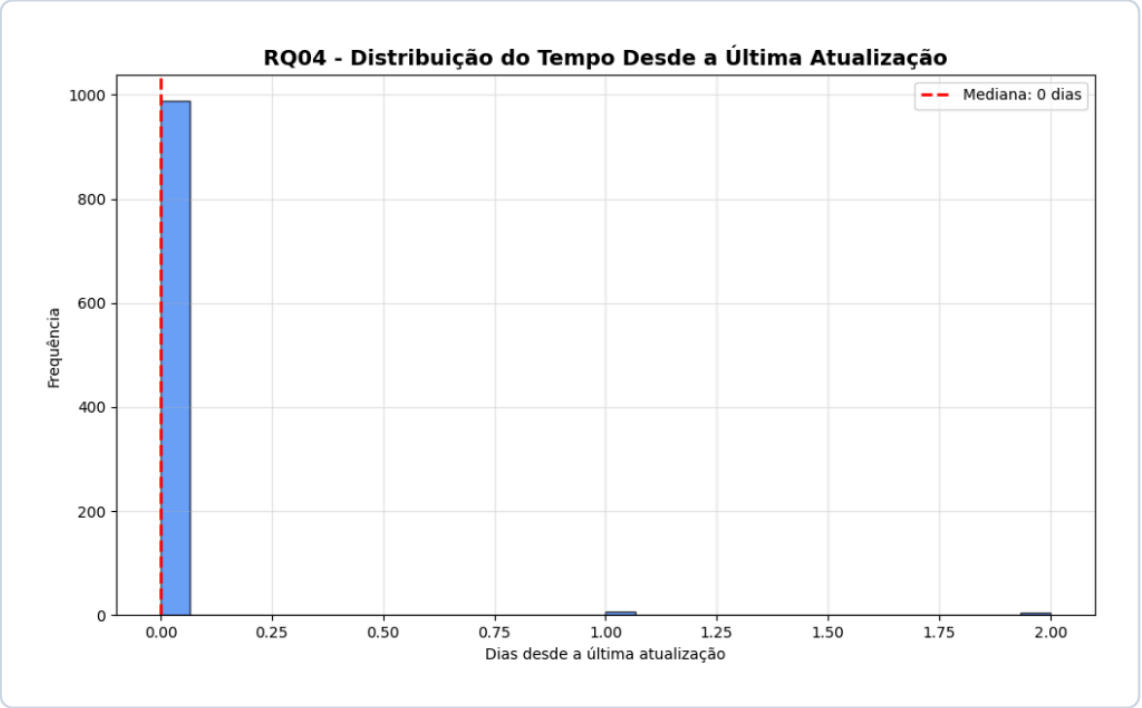


Descrição do gráfico: O gráfico apresenta a distribuição do número de pull requests aceitos pelos repositórios analisados. Observa-se uma distribuição altamente assimétrica à direita, indicando que a maioria dos repositórios possui um número moderado de contribuições, enquanto alguns poucos projetos concentram volumes extremamente elevados de pull requests. A linha tracejada vermelha indica a **mediana de 739 pull requests aceitos**. Esse padrão sugere que projetos populares tendem a receber colaboração significativa da comunidade, embora a intensidade dessa colaboração varie amplamente entre os projetos.

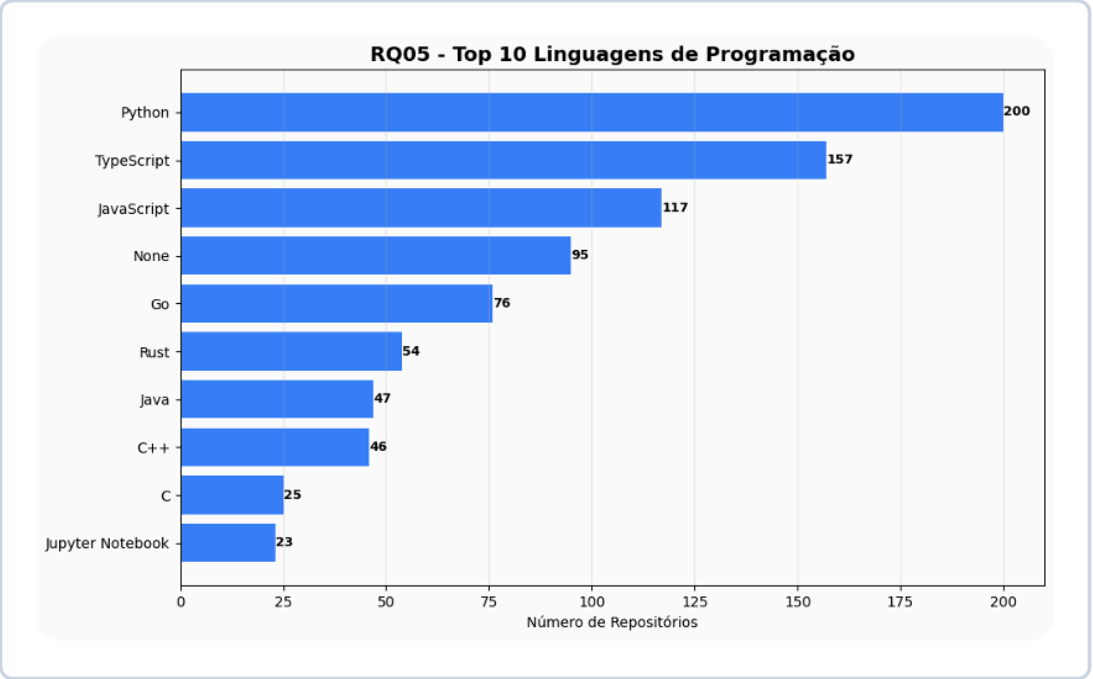


Descrição do gráfico: O histograma do número de releases mostra a frequência com que os repositórios populares publicam versões oficiais de seus softwares. A distribuição apresenta forte

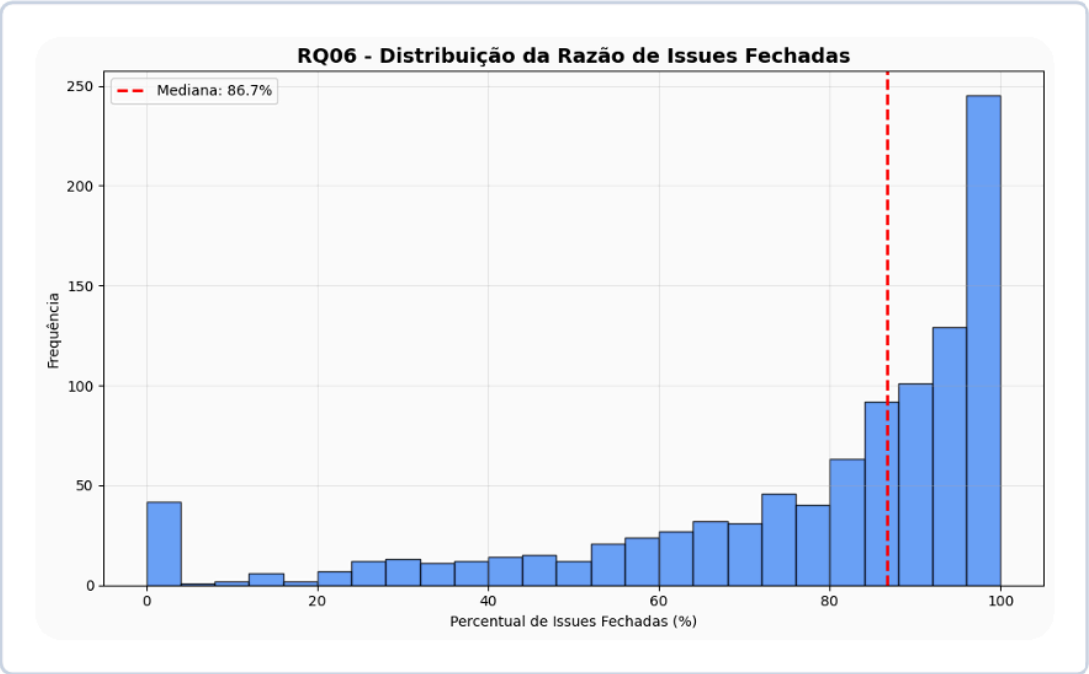
assimetria positiva, indicando que muitos projetos possuem um número relativamente baixo ou moderado de releases, enquanto alguns projetos mantêm ciclos de lançamento muito mais intensos. A linha tracejada indica uma mediana de 40 releases, o que sugere que os projetos populares tendem a estruturar e formalizar suas entregas por meio de versões publicadas regularmente, refletindo um processo de desenvolvimento ativo.



Descrição do gráfico: O histograma do tempo desde a última atualização apresenta a distribuição da quantidade de dias decorridos entre a última atividade de commit e a data da coleta dos dados. A figura mostra uma alta concentração de repositórios na primeira faixa (0 dias), indicando que a grande maioria dos projetos populares havia sido atualizada extremamente recentemente no momento da extração dos dados. A linha tracejada vermelha, representando a mediana de 0 dias, confirma que a manutenção ativa é uma característica predominante nesses projetos, validando a hipótese de que sistemas populares são atualizados com frequência.



Descrição do gráfico: O gráfico de barras apresenta as dez linguagens de programação mais utilizadas entre os repositórios analisados. Observa-se a predominância de **Python**, seguida por **TypeScript** e **JavaScript**, indicando forte presença de tecnologias associadas ao desenvolvimento web, ciência de dados e inteligência artificial. Outras linguagens relevantes incluem **Go, Rust, Java, C++ e C**, que também possuem grande adoção em sistemas de infraestrutura e software de alto desempenho. A presença dessas linguagens sugere que projetos populares tendem a ser desenvolvidos em tecnologias amplamente adotadas pela comunidade de software.



Descrição do gráfico: O histograma da razão de issues fechadas apresenta a distribuição do percentual de issues resolvidas em relação ao total de issues registradas em cada repositório. Observa-se uma forte concentração de valores nas faixas mais altas do gráfico, indicando que grande parte dos projetos possui taxas elevadas de resolução de issues. A linha tracejada vermelha marca a mediana de aproximadamente 86,7%, sugerindo que os repositórios populares mantêm uma boa capacidade de resposta na resolução de problemas, correção de bugs e gerenciamento das demandas da comunidade.

4.1 Discussão dos Resultados

4.1.1 Confrontar Questões-Pesquisa

- Os resultados obtidos permitem confrontar as questões-pesquisa (Seção 1.3) da seguinte forma:

- **RQ01. Sistemas populares são maduros/antigos?**
  Sim. A mediana de idade foi de 3090 dias, o equivalente a aproximadamente 8,5 anos, indicando que os repositórios mais populares são, em sua maioria, projetos maduros e consolidados ao longo do tempo.

- **RQ02. Sistemas populares recebem muita contribuição externa?**
  Sim. A mediana de 739 pull requests aceitos sugere que esses projetos recebem participação expressiva da comunidade, evidenciando abertura à colaboração externa.

- **RQ03. Sistemas populares lançam releases com frequência?**
  Sim. A mediana de 40 releases indica que os projetos populares tendem a estruturar e

formalizar suas entregas ao longo do tempo, o que sugere governança e evolução contínua do software.

- **RQ04. Sistemas populares são atualizados com frequência?**
  Sim. A mediana de 0 dias desde a última atualização mostra que, no momento da coleta, uma parcela significativa desses repositórios havia sido atualizada muito recentemente, caracterizando manutenção ativa.

- **RQ05. Sistemas populares são escritos nas linguagens de programação mais populares?**
  Sim. As linguagens predominantes foram Python (20,0%), TypeScript (15,7%) e JavaScript (11,7%), todas amplamente difundidas e associadas a ecossistemas muito ativos.

- **RQ06. Sistemas populares possuem um alto percentual de issues fechadas?**
  Sim. A mediana da razão de issues fechadas foi de 86,7%, valor que reforça a percepção de que esses projetos possuem boa capacidade de resposta e manutenção.

4.1.2 Confrontar Hipóteses

Os resultados obtidos permitem avaliar as hipóteses formuladas na Seção 1.4 da seguinte forma:

- **Hipótese Principal: Repositórios populares apresentam características positivas de manutenção e colaboração.**
  Aceita. Os resultados indicam que os repositórios analisados apresentam, em geral, alto nível de maturidade, forte participação da comunidade e manutenção ativa. A mediana de idade foi de aproximadamente 8,5 anos, a mediana de pull requests aceitos foi de 739 e a mediana de releases foi de 40. Além disso, observou-se atualização recente dos projetos e predominância de linguagens amplamente utilizadas no mercado.

- **H1. Repositórios populares tendem a ser projetos de longa data.**
  Aceita. A mediana de idade dos repositórios foi de 3090 dias (aproximadamente 8,5 anos), indicando que a maioria dos projetos populares possui longo histórico de desenvolvimento e maturação.

- **H2. Sistemas populares recebem muita contribuição externa.**
  Aceita. A mediana de 739 pull requests aceitos sugere um volume expressivo de contribuições externas, evidenciando participação ativa da comunidade no desenvolvimento desses projetos.

- **H3. Projetos populares realizam releases com frequência.**
  Aceita. A mediana de 40 releases indica que os projetos populares mantêm um processo estruturado de lançamento de versões, sinalizando evolução contínua e organização no desenvolvimento.

- **H4. Sistemas populares são atualizados com frequência.**
  Aceita. A mediana de 0 dias desde a última atualização indica que muitos dos repositórios

analisados haviam sido atualizados muito recentemente no momento da coleta, evidenciando manutenção ativa.

- **H5. Sistemas populares utilizam linguagens de programação amplamente difundidas.**
  Aceita. As linguagens mais frequentes entre os repositórios analisados foram Python, TypeScript e JavaScript, todas amplamente utilizadas e associadas a grandes comunidades de desenvolvedores.

- **H6. Sistemas populares possuem alta taxa de resolução de issues.**
  Aceita parcialmente. A mediana da taxa de issues fechadas foi de 86,7%, indicando boa capacidade de manutenção. Entretanto, o teste estatístico mostrou que a média observada (0,77) é estatisticamente diferente do valor esperado de 0,8 ($p = 0,000614$), sugerindo que, embora a taxa seja elevada, ela não corresponde exatamente ao valor hipotetizado.

4.2 Insights

Os resultados do experimento fornecem os seguintes *insights* importantes para a área de estudo:

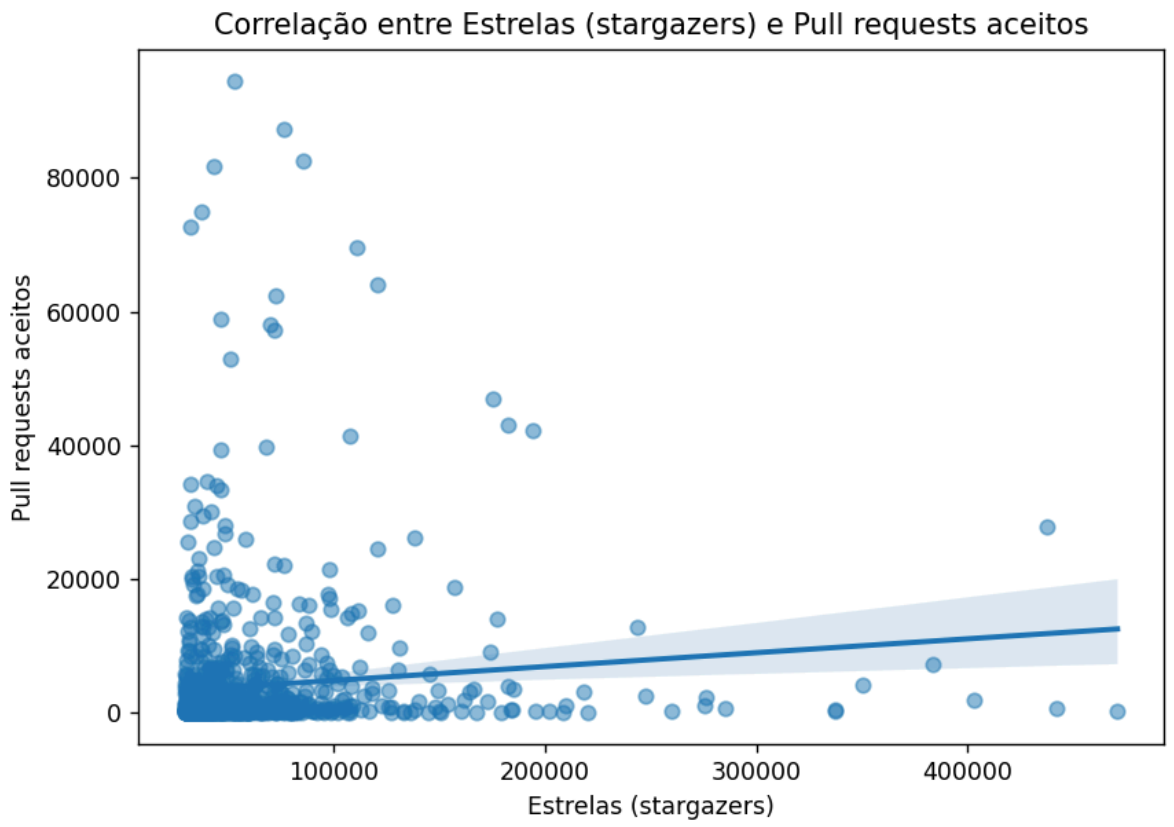
- **Insight 1:**
  Os resultados apontam alguns insights relevantes. Primeiro, a popularidade parece estar associada à maturidade do projeto, já que os repositórios analisados apresentam idade mediana elevada. Isso sugere que confiança, adoção e reconhecimento da comunidade são fenômenos que tendem a se consolidar ao longo do tempo.

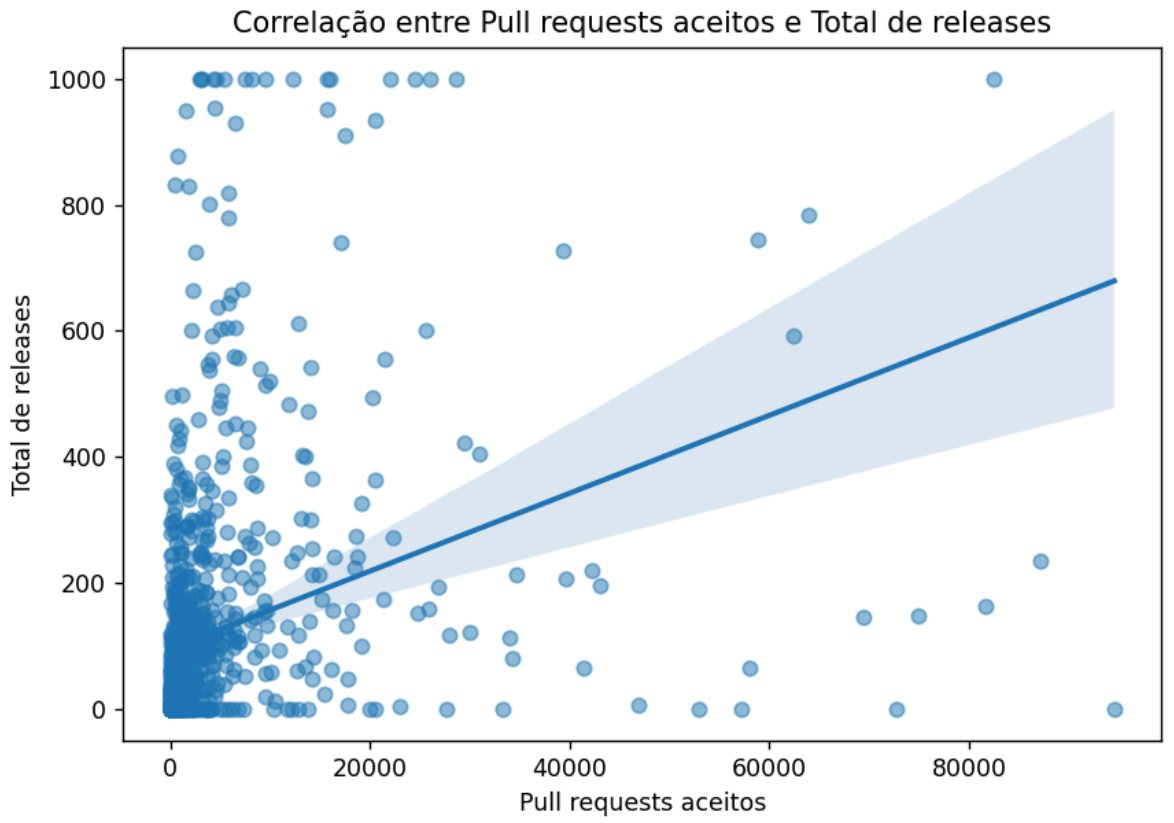
- **Insight 2:**
  Segundo, os dados indicam que projetos populares costumam apresentar forte colaboração externa, evidenciada pelo elevado número de pull requests aceitos. Isso sugere que a capacidade de atrair contribuidores e incorporar mudanças da comunidade é um elemento importante na sustentabilidade desses projetos.

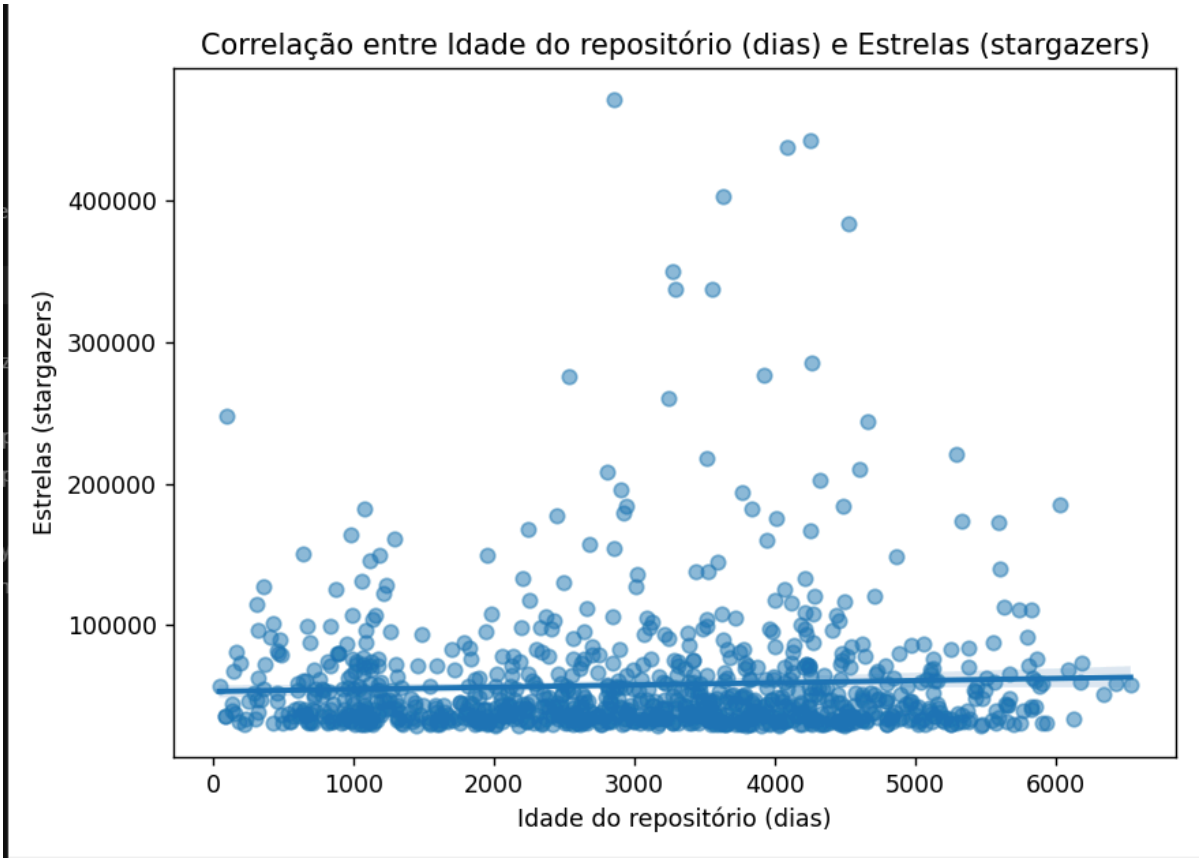
- **Insight 3:**
  Terceiro, observa-se forte sinal de manutenção ativa, tanto pela mediana de 0 dias desde a última atualização quanto pela razão mediana de 86,7% de issues fechadas. Em conjunto, esses indicadores sugerem que repositórios populares não apenas atraem usuários, mas também continuam recebendo atenção constante de seus mantenedores.

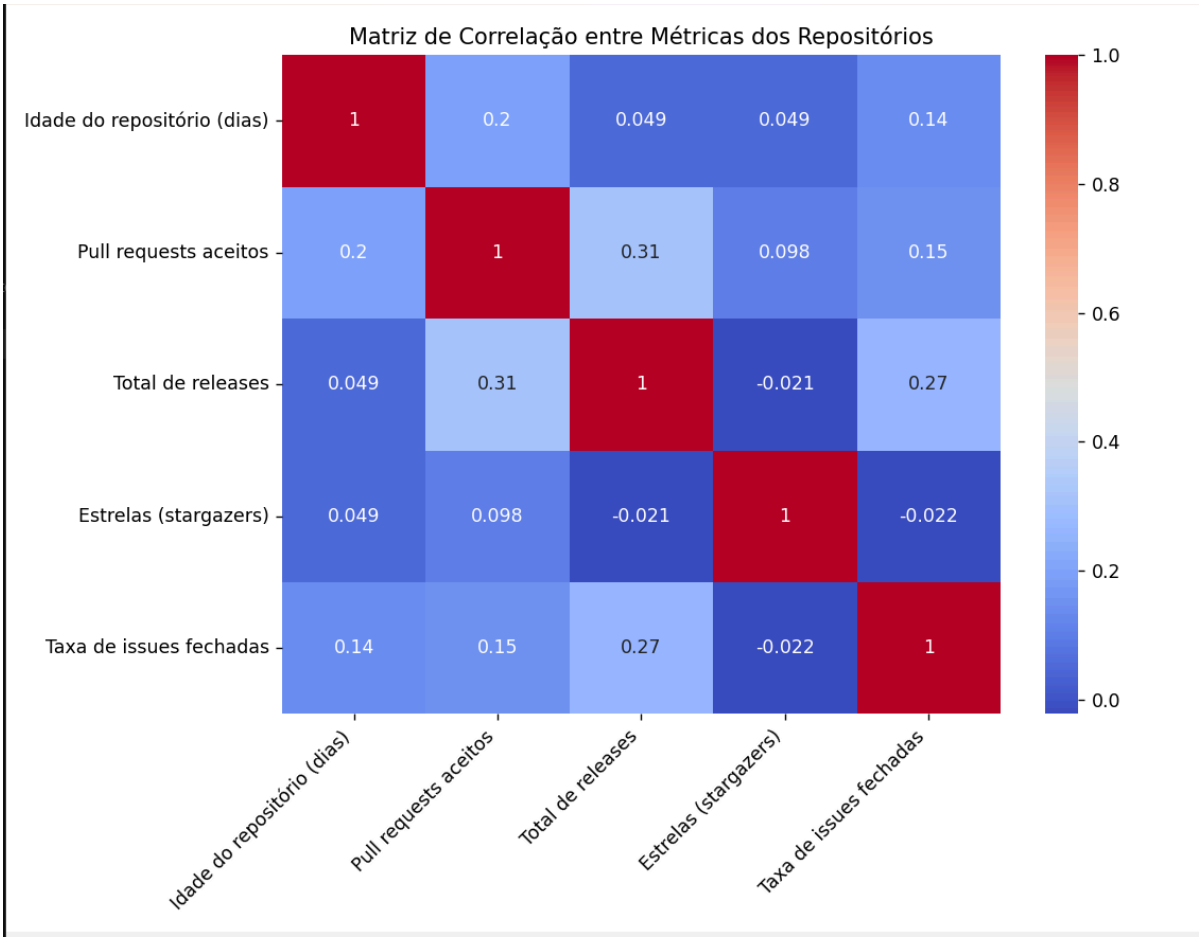
  Por fim, a predominância de linguagens como Python, TypeScript e JavaScript sugere que a popularidade dos repositórios também está relacionada a linguagens com ecossistemas amplos, alta adoção e grande disponibilidade de desenvolvedores.

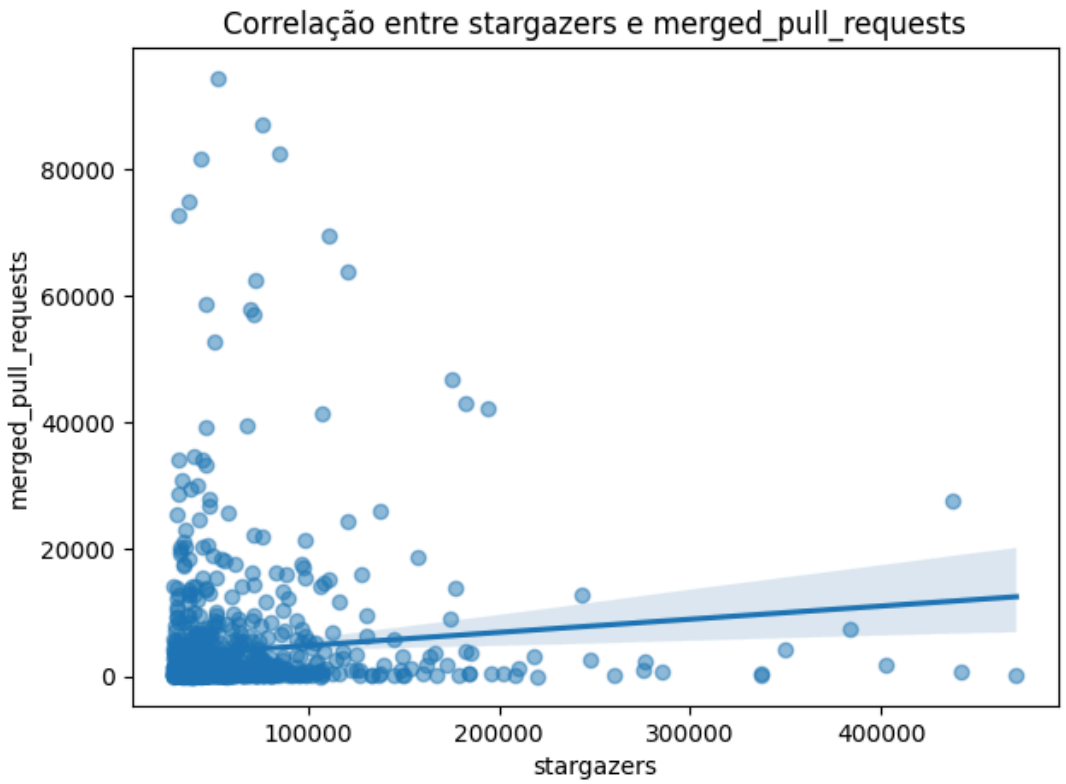
4.3 Gráficos


Correlação entre Estrelas (stargazers) e Pull requests aceitos
Pull requests aceitos
0
20000
40000
60000
80000
100000
200000
300000
400000
Estrelas (stargazers)


Correlação entre Pull requests aceitos e Total de releases
Total de releases
0
200
400
600
800
1000
0
20000
40000
60000
80000
Pull requests aceitos


Correlação entre Idade do repositório (dias) e Estrelas (stargazers)
Estrelas (stargazers)
400000
300000
200000
100000
0
1000
2000
3000
4000
5000
6000
Idade do repositório (dias)


Matriz de Correlação entre Métricas dos Repositórios
Idade do repositório (dias)
Pull requests aceitos
Total de releases
Estrelas (stargazers)
Taxa de issues fechadas
1
0.2
0.049
0.049
0.14
0.2
1
0.31
0.098
0.15
0.049
0.31
1
-0.021
0.27
0.049
0.098
-0.021
1
-0.022
0.14
0.15
0.27
-0.022
1
1.0
0.8
0.6
0.4
0.2
0.0


Correlação entre stargazers e merged_pull_requests
merged_pull_requests
80000
60000
40000
20000
0
100000
200000
300000
400000
stargazers

4.4 Comparações

Ao comparar os resultados obtidos com estudos anteriores sobre mineração de repositórios de software (MSR), observa-se que os padrões encontrados são semelhantes aos relatados na literatura. Projetos populares tendem a apresentar maior maturidade, elevada atividade de desenvolvimento e forte participação da comunidade. Os resultados deste experimento indicam que os repositórios analisados possuem idade média superior a oito anos, alta quantidade de pull requests aceitos e ciclos frequentes de lançamento de versões. Além disso, a análise de correlação sugere que projetos com maior volume de contribuições também tendem a apresentar maior frequência de releases, o que está alinhado com estudos que associam atividade de desenvolvimento à evolução contínua do software.

4.5 Estatísticas

As principais estatísticas descritivas e inferenciais obtidas a partir da análise dos 1.000 repositórios mais populares do GitHub são apresentadas a seguir.

A tabela a seguir sumariza as medidas de tendência central e dispersão para as principais métricas quantitativas do experimento.

Métrica	Média	Mediana (Métrica de Decisão)	Intervalo de Confiança (95%)	Unidade	Assimetria Comentário
Idade do Repositório (RQ01)	2999,28	3089,50	[2906,99 ; 3091,57]	Dias	Média e Mediana estão próximas, indicando uma distribuição razoavelmente normal, confirmando a maturidade geral dos projetos.
Pull Requests Aceitos (RQ02)	3947,13	739	[3336,51 ; 4557,75]	Quant.	A grande discrepância entre Média e Mediana (3947 vs. 739) revela forte assimetria positiva, indicando que a Média é puxada para cima por um pequeno número de projetos com volumes *outliers* (muito elevados) de contribuições.
Número de Releases (RQ03)	120,08	40	[107,73 ; 132,43]	Quant.	Também apresenta assimetria, com a Mediana significativamente menor que a Média, mas reforça a atividade frequente de lançamento de versões.

Razão de Issues Fechadas (RQ06)	0,77 (77%)	0,87 (87%)	[0,76 ; 0,79]	%	A Mediana (87%) confirma a Hipótese H6 (acima de 80%). A Média (77%) e o Teste t subsequente indicam que o desempenho médio de resolução é inferior ao esperado, refletindo a influência de projetos com baixa taxa de resolução no conjunto total de dados.

- A idade dos repositórios apresentou média de 2999,28 dias e mediana de 3089,50 dias, com intervalo de confiança de 95% entre 2906,99 e 3091,57 dias, indicando projetos maduros.
- Os pull requests aceitos apresentaram média de 3947,13 e mediana de 739, com intervalo de confiança de 95% entre 3336,51 e 4557,75. A diferença entre média e mediana mostra forte assimetria, causada por projetos com volume muito elevado de contribuições.
- O número de releases teve média de 120,08 e mediana de 40, com intervalo de confiança de 95% entre 107,73 e 132,43, sugerindo atividade frequente de lançamento de versões.
- A razão de issues fechadas apresentou média de 0,77 e mediana de 0,87, com intervalo de confiança de 95% entre 0,76 e 0,79. Apesar da mediana elevada, o teste t resultou em $t = -3,4363$ e $p = 0,000614$, indicando que a média ficou significativamente abaixo de 0,80.
- Na correlação de Pearson, observou-se relação fraca positiva entre estrelas e pull requests aceitos ($r = 0,098$; $p = 0,002$), relação não significativa entre idade e número de estrelas ($r = 0,049$; $p = 0,119$) e relação moderada positiva entre pull requests aceitos e releases ($r = 0,305$; $p < 0,001$).

5. Conclusão

5.1 Tomada de decisão

Com base nos resultados obtidos, pode-se concluir que a hipótese principal do experimento é parcialmente confirmada. Os dados indicam que repositórios populares do GitHub apresentam características consistentes de maturidade, colaboração ativa da comunidade e manutenção frequente. A análise mostrou que esses projetos possuem idade média superior a oito anos, elevado volume de pull requests aceitos e ciclos regulares de lançamento de versões. Além disso, a mediana da razão de issues fechadas foi superior a 80%, indicando boa capacidade de manutenção e resposta às demandas da comunidade.

Entretanto, a análise inferencial indicou que a média da taxa de issues fechadas foi inferior ao limiar de 80%, sugerindo que nem todos os projetos mantêm níveis igualmente elevados de resolução de issues. Dessa forma, os resultados reforçam que projetos populares tendem a apresentar boas práticas de manutenção e colaboração, mas também evidenciam variações significativas entre os repositórios analisados.

5.2 Sugestões futuras

Para complementar o estudo ou mitigar as limitações encontradas, sugere-se:

- Analisar um número maior de repositórios.
- Comparar projetos populares com projetos menos populares.
- Investigar outros fatores, como número de contribuidores, frequência de commits e atividade da comunidade.

5.3 Apresentar resultado conclusivo sucinto

Em resumo, o experimento demonstrou que os repositórios mais populares do GitHub apresentam padrões claros de maturidade, colaboração ativa da comunidade e manutenção frequente, características que parecem estar associadas à popularidade desses projetos.